

TECHNICAL DEEP-DIVE & ARCHITECTURE MANUAL

Conversational RAG Chatbot with Hybrid Retrieval & Local AI Pipelines

1. Core AI Models Implemented

To achieve offline independence and high precision across government rules, a combination of local GPU models, remote specialized endpoints, and local CPU inference models is utilized:

- **Qwen/Qwen3-VL-Embedding-2B (Local GPU):** Responsible for generating the 2048-dimensional dense vector embeddings. It captures semantic, contextual, and conceptual representations of the rules chunks, enabling meaning-based similarity retrieval.
- **Gemma-4-Vision (Port 3001 - Remote VLM):** Operates as the vision-based optical character recognition (OCR) fallback. When a PDF page has low text density (<50 characters), the ingestion pipeline renders the page image and queries Gemma-4-Vision to transcribe text from scanned circular templates.
- **Sarvam-105B (Port 3002 - Remote LLM):** Used for high-throughput zero-shot category classification in the document sorting pipeline (segregate.py). Running on SGLang, it isolates the reasoning process natively in a separate reasoning_content field, delivering clean classifications in under 3.6 seconds.
- **Ollama Llama3.2 / Qwen3 (Local Host CPU/GPU):** Handles general question-answering and synthesis of RAG context. The RAG chain passes the top-ranked retrieved context chunks and chat history to the LLM to generate the final response.

2. Algorithmic & Retrieval Techniques

- **Dual-Vector Dense-Sparse Hybrid Search:** The Qdrant collection is configured to hold both named dense vectors and sparse vector indices. Every document chunk is indexed with both representations in a single database point, allowing single-query native search execution.
- **Deterministic Adler32 Hashing Trick (Sparse Vectorizer):** To compute sparse vectors on-the-fly, a custom HashingSparseEncoder maps terms to sparse indices using Adler32 checksums. This allows the query engine and ingestion scripts to resolve terms to identical dimensions deterministically, saving megabytes of vocabulary files on disk.
- **Native Reciprocal Rank Fusion (RRF):** Merges dense semantic ranks and sparse term-match ranks natively in Qdrant, ensuring that documents containing exact rule codes (e.g. 'FR 49') and conceptual context are merged into a single relevancy list.

- **Dynamic Cosine Fallback Routing:** If the top matched chunk from a specific folder search yields a cosine similarity score below 0.70, the query retriever automatically relaxes the search scope filter to the parent category or globally, preventing false negatives caused by incorrect directory sorting.
- **Contextual Metadata Breadcrumbs:** Before vectorization, each chunk is prepended with its relative directory location (e.g., [Finance -> Pay & Increments]). This injects critical hierarchical context directly into the text embeddings, improving matching accuracy.

3. Engineering Safeguards & Performance

To run reliably in production environments, several safeguards were implemented:

- **Database Lock Prevention:** The pipeline automatically stops active Streamlit instances before rebuilding Qdrant to release SQLite database locks.
- **GPU VRAM Garbage Collection:** Wrapped chunk commits in try-finally blocks during ingestion to clear the queue, preventing CUDA Out-of-Memory (OOM) errors during large batch commits.
- **Warm-up Cache Singleton:** Saves the query chain structure in a global cache, reducing subsequent query latencies by 4.6x (from 16.7s to 3.6s).